

Menu-Driven Square and Rectangular Patterns

Using Switch Statement, Functions and Nested Loops

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Aim

To write and test a menu-driven C program that uses a switch statement, nested loops and separate parameterless functions to print basic square and rectangular patterns without using arrays.

2. Problem Statement

Display a menu containing four pattern choices. According to the user's choice, print the selected pattern using nested loops. The menu provides: solid square, solid rectangle, row-number pattern and column-number pattern. Each pattern is implemented in a separate function with no parameters and no return value.

3. Concepts Used

- switch statement for menu selection
- do-while loop to display the menu repeatedly
- nested for loops for rows and columns
- parameterless void functions
- input validation for positive row and column values
- no arrays

4. Menu

1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit

5. Algorithm

1. Start.
2. Declare choice and function prototypes.
3. Use a do-while loop to display the menu.
4. Read the user's choice.
5. Use switch(choice) to select the required operation.
6. For choice 1, call solidSquare(). Input positive n and print n rows and n columns of * using nested loops.
7. For choice 2, call solidRectangle(). Input positive m and n and print m rows and n columns of * using nested loops.
8. For choice 3, call rowNumberPattern(). Input positive m and n. For each row i, print i exactly n times.

9. For choice 4, call `columnNumberPattern()`. Input positive `m` and `n`. In every row, print column numbers 1 to `n`.
10. For choice 5, display an exit message.
11. For any other choice, display an invalid-choice error message.
12. Repeat the menu until choice 5 is selected.
13. Stop.

6. Test-Case Table

TC	Input / Choice	Type	Expected Result
1	Choice 1, n=4	Valid	4 x 4 solid square of *
2	Choice 2, m=3, n=5	Valid	3 x 5 solid rectangle of *
3	Choice 3, m=4, n=4	Valid	Rows contain 1s, 2s, 3s and 4s respectively
4	Choice 4, m=4, n=4	Valid	Each row prints 1 2 3 4
5	Choice 1, n=1	Boundary	Single * is printed
6	Choice 2, m=1, n=1	Boundary	Single * is printed
7	Choice 1, n=0 then 3	Invalid -> valid	Error for 0; accept 3 and print 3 x 3 square
8	Choice 2, m=-2 then 2, n=3	Invalid -> valid	Error for negative rows; then print 2 x 3 rectangle
9	Choice 9	Invalid menu	Display Invalid menu choice message
10	Choice 5	Exit	Program terminates normally

7. C Program

```
#include <stdio.h>

void solidSquare(void);
void solidRectangle(void);
void rowNumberPattern(void);
void columnNumberPattern(void);

int main(void)
{
    int choice;

    do
    {
        printf("\n===== PATTERN MENU =====\n");
        printf("1. Solid square pattern\n");
        printf("2. Solid rectangular pattern\n");
        printf("3. Row-number pattern\n");
        printf("4. Column-number pattern\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: solidSquare(); break;
            case 2: solidRectangle(); break;
            case 3: rowNumberPattern(); break;
            case 4: columnNumberPattern(); break;
            case 5: printf("Exiting program...\n"); break;
            default: printf("Error: Invalid menu choice.\n");
        }
    } while (choice != 5);

    return 0;
}

void solidSquare(void)
{
    int n, i, j;
    do {
```

```

        printf("Enter number of rows/columns (positive): ");
        scanf("%d", &n);
        if (n <= 0) printf("Error: Enter a positive integer.\n");
    } while (n <= 0);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) printf("* ");
        printf("\n");
    }
}

void solidRectangle(void)
{
    int m, n, i, j;
    do {
        printf("Enter rows m (positive): "); scanf("%d", &m);
        if (m <= 0) printf("Error: Enter a positive integer.\n");
    } while (m <= 0);
    do {
        printf("Enter columns n (positive): "); scanf("%d", &n);
        if (n <= 0) printf("Error: Enter a positive integer.\n");
    } while (n <= 0);

    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++) printf("* ");
        printf("\n");
    }
}

void rowNumberPattern(void)
{
    int m, n, i, j;
    do {
        printf("Enter rows m (positive): "); scanf("%d", &m);
        if (m <= 0) printf("Error: Enter a positive integer.\n");
    } while (m <= 0);
    do {
        printf("Enter columns n (positive): "); scanf("%d", &n);
        if (n <= 0) printf("Error: Enter a positive integer.\n");
    } while (n <= 0);

    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++) printf("%d ", i);
        printf("\n");
    }
}

void columnNumberPattern(void)
{
    int m, n, i, j;
    do {
        printf("Enter rows m (positive): "); scanf("%d", &m);
        if (m <= 0) printf("Error: Enter a positive integer.\n");
    } while (m <= 0);
    do {
        printf("Enter columns n (positive): "); scanf("%d", &n);
        if (n <= 0) printf("Error: Enter a positive integer.\n");
    } while (n <= 0);
}

```

```
for (i = 1; i <= m; i++) {  
    for (j = 1; j <= n; j++) printf("%d ", j);  
    printf("\n");  
}
```

8. Compilation and Execution

```
gcc day23.c -o day23
./day23
```

The program was checked with the prepared valid, invalid and boundary test cases. The following sample runs show the expected output for each activity.

9. Sample Output / Execution

9.1 Solid Square Pattern

```
Enter your choice: 1
Enter number of rows/columns (positive): 4
* * * *
* * * *
* * * *
* * * *
```

9.2 Solid Rectangular Pattern

```
Enter your choice: 2
Enter rows m (positive): 3
Enter columns n (positive): 5
* * * * *
* * * * *
* * * * *
```

9.3 Row-Number Pattern

```
Enter your choice: 3
Enter rows m (positive): 4
Enter columns n (positive): 4
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4
```

9.4 Column-Number Pattern

```
Enter your choice: 4
Enter rows m (positive): 4
Enter columns n (positive): 4
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
```

9.5 Invalid Choice and Exit

```
Enter your choice: 9
Error: Invalid menu choice.
```

```
Enter your choice: 5
Exiting program...
```

10. Explanation

The outer loop controls the rows and the inner loop controls the columns. In the solid patterns, the inner loop prints an asterisk. In the row-number pattern, the row variable *i* is printed repeatedly. In the column-number pattern, the column variable *j* is printed. The switch statement calls the required function according to the menu choice.

11. Result

Thus, the menu-driven C program for printing solid square, solid rectangular, row-number and column-number patterns was written using a switch statement, parameterless void functions and nested loops without arrays. The program handles invalid menu choices and non-positive dimensions and produces the required patterns for the prepared test cases.